

MNIST Handwritten Digit Recognition

MNIST Data Details

Dataset Name : MNIST (Modified National Institute of Standards and Technology)

Total Images : 70,000

Training Images : 60,000

Test Images : 10,000

Image Size : 28 × 28 pixels

Color Channels : 1 (Grayscale)

Number of Classes : 10

Images per Class ~ 7,000 (slight variations per class)

Data Type : Grayscale Images

Task Type : Image Classification / Handwritten Digit Recognition

```
In [25]: import tensorflow as tf
         from tensorflow import keras
         from tensorflow.keras import layers
         import matplotlib as plt
```

```
In [26]: (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data
```

```
In [27]: x_train = x_train/255
         x_test = x_test/255
```

```
In [28]: model = keras.Sequential([
        layers.Conv2D(28, (3,3), activation = 'relu', input_shape = (28,
        layers.MaxPooling2D((2,2)),

        layers.Conv2D(56, (3,3), activation = 'relu'),
        layers.MaxPooling2D((2,2)),

        layers.Dropout(0.25),

        layers.Flatten(),
        layers.Dense(512, activation = 'relu'),
        layers.Dropout(0.5),
```

```
layers.Dense(10, activation= 'softmax')
])
```

```
In [29]: model.compile(
    optimizer = 'adam',
    loss = 'sparse_categorical_crossentropy',
    metrics = ['accuracy']
)
```

```
In [30]: history = model.fit(
    x_train, y_train,
    epochs = 7,
    batch_size = 64,
    validation_data = (x_test, y_test)
)
```

Epoch 1/7

938/938  **25s** 26ms/step - accuracy: 0.9408 - loss: 0.1904 - val_accuracy: 0.9859 - val_loss: 0.0446

Epoch 2/7

938/938  **22s** 23ms/step - accuracy: 0.9801 - loss: 0.0642 - val_accuracy: 0.9888 - val_loss: 0.0366

Epoch 3/7

938/938  **20s** 22ms/step - accuracy: 0.9847 - loss: 0.0483 - val_accuracy: 0.9922 - val_loss: 0.0272

Epoch 4/7

938/938  **25s** 27ms/step - accuracy: 0.9882 - loss: 0.0382 - val_accuracy: 0.9914 - val_loss: 0.0256

Epoch 5/7

938/938  **30s** 32ms/step - accuracy: 0.9896 - loss: 0.0317 - val_accuracy: 0.9921 - val_loss: 0.0241

Epoch 6/7

938/938  **30s** 32ms/step - accuracy: 0.9908 - loss: 0.0290 - val_accuracy: 0.9928 - val_loss: 0.0220

Epoch 7/7

938/938  **27s** 29ms/step - accuracy: 0.9919 - loss: 0.0264 - val_accuracy: 0.9927 - val_loss: 0.0239

```
In [31]: test_loss, test_acc = model.evaluate(x_test, y_test)
    print("Test_accuracy: ", test_acc)
```

313/313  **1s** 4ms/step - accuracy: 0.9927 - loss: 0.0239

Test_accuracy: 0.9926999807357788

```
In [32]: print(y_train.shape)
    print(y_test.shape)

    print(y_train[:10])
    print(y_test[:10])
```

(60000,)

(10000,)

[5 0 4 1 9 2 1 3 1 4]

[7 2 1 0 4 1 4 9 5 9]

```
In [33]: pred = model.predict(x_test[:5])

print(pred)
print(pred.sum(axis=1))
```

1/1 ————— 0s 51ms/step
[[3.5614216e-09 3.5092242e-09 7.5775219e-08 2.5720108e-09 2.1299577e-10
7.3332326e-11 3.3377465e-14 9.9999988e-01 9.0984636e-10 8.1825062e-09]
[9.7035425e-08 8.8749479e-09 9.9999988e-01 1.8785251e-12 1.8256182e-12
1.5062196e-15 7.9679499e-09 4.5154158e-12 3.9562617e-11 8.8018751e-15]
[2.9683178e-09 9.9997205e-01 5.1170119e-08 1.2015658e-10 1.0220502e-05
5.9307901e-08 1.2398011e-08 1.7637714e-05 7.1321544e-08 3.0883307e-08]
[9.9999952e-01 9.2333797e-12 6.2298344e-10 1.6180941e-10 1.8384892e-10
4.0709565e-09 4.8083882e-07 1.5944648e-09 2.5649221e-08 2.6812701e-09]
[9.4880313e-08 1.2350150e-08 4.6739466e-09 4.8142790e-10 9.9995518e-01
3.0530675e-08 1.0005201e-07 2.0614634e-08 2.4482329e-06 4.2088177e-05]]
[0.99999994 1. 1.00000001 1. 1.]]

```
In [34]: print(x_train.shape)
print(x_test.shape)
```

```
(60000, 28, 28)
(10000, 28, 28)
```

```
In [43]: import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
# If using Google Colab, uncomment the next two lines to upload dir
# from google.colab import files
# uploaded = files.upload()

# 1. Define the path to your uploaded image
# (Replace 'my_digit.jpg' with your actual filename)
image_path = 'my_digit.jpg'

# 2. Load the image, convert to grayscale, and resize to 28x28
img = image.load_img(image_path, color_mode="grayscale", target_size=(28, 28))

# 3. Convert the image to a numpy array
img_array = image.img_to_array(img)

# 4. Preprocess the image
# MNIST digits are white on a black background. If your uploaded image is
# dark ink on white paper, uncomment the next line to invert the color
# img_array = 255.0 - img_array

# Normalize the pixel values to [0, 1] just like your training data
```

```
img_array = img_array / 255.0

# 5. Convert the array explicitly into a TensorFlow Tensor and set
img_tensor = tf.convert_to_tensor(img_array, dtype=tf.float32)
img_input = tf.expand_dims(img_tensor, axis=0) # Shape becomes (1,

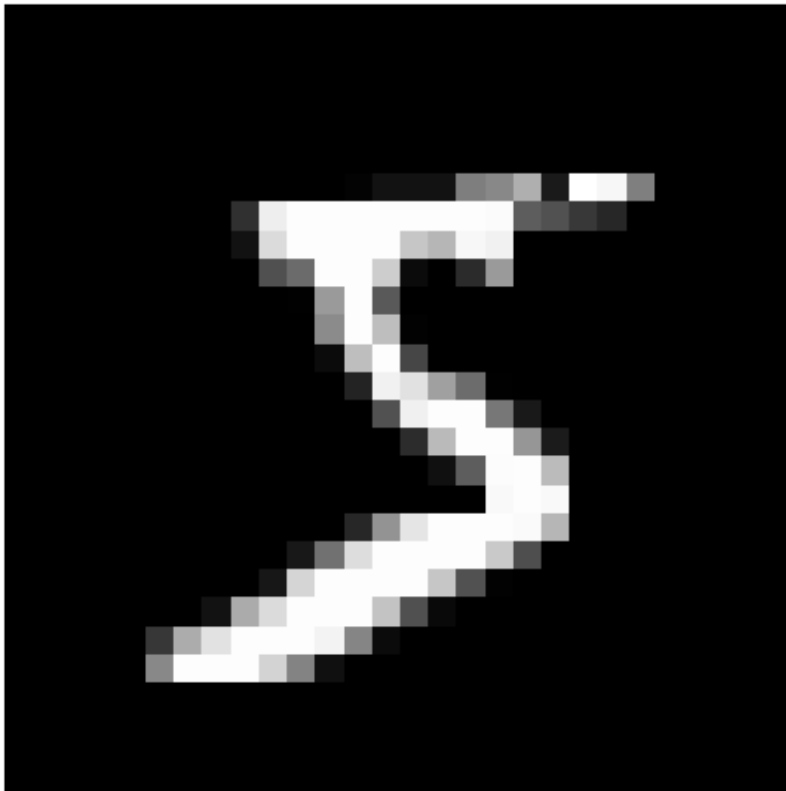
# 6. Make the prediction using direct call syntax (bypasses model.p
predictions = model(img_input, training=False).numpy()

# If your final layer does NOT use softmax (Option 1 from earlier),
# uncomment the next line to convert raw scores (logits) into proba
# import tensorflow as tf
# predictions = tf.nn.softmax(predictions).numpy()

predicted_class = np.argmax(predictions)
confidence = np.max(predictions) * 100

# 7. Display the image and the model's guess
plt.imshow(img_array.reshape(28, 28), cmap='gray')
plt.title(f"Prediction: {predicted_class} ({confidence:.2f}% Confid
plt.axis('off')
plt.show()
```

Prediction: 5 (99.23% Confidence)



In []: